

Introduction to ROS2

June 3rd, 2026

Author:

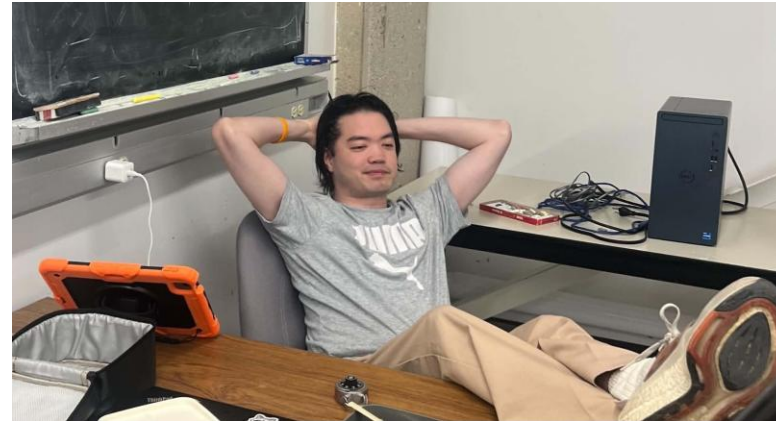
Pi Thanacha Choopojcharoen →

Presented by:

Ramisha Anjum



UNIVERSITY OF
WATERLOO



Agenda

- ROS
- ROS2
- Nodes & Topics
- Interfaces
 - Message
 - Services (*)
 - Actions (*)
- Packages

Main Objective:

To read the RCLPY-based code, create and customize basic ROS2 node.

Motivation

Distributed Publish-Subscribe Middleware



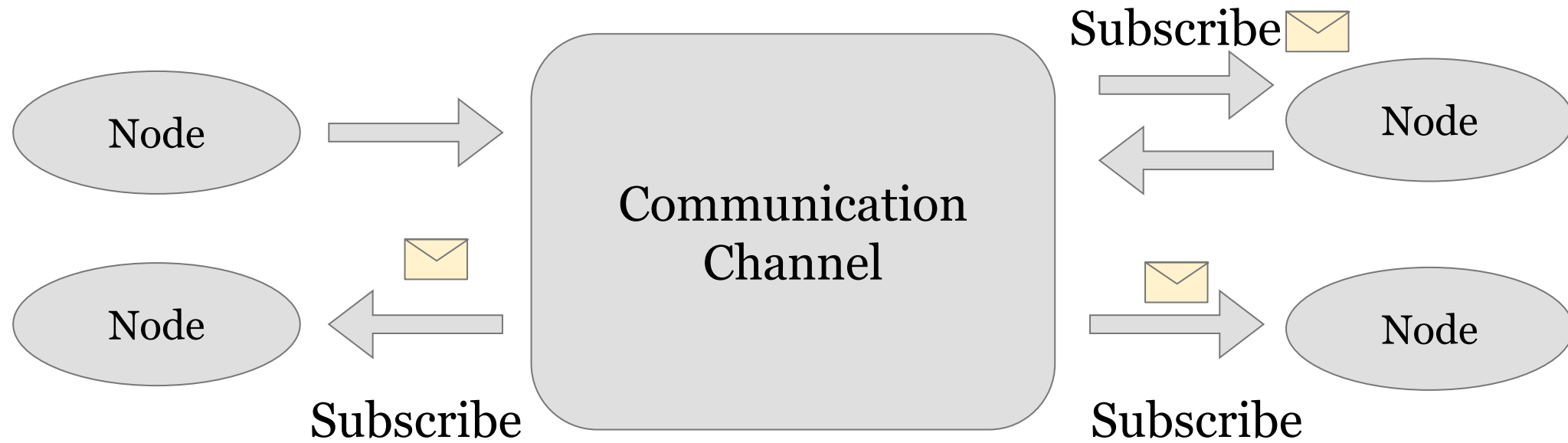
Distributed Publish-Subscribe Middleware



Distributed Publish-Subscribe Middleware



Distributed Publish-Subscribe Middleware

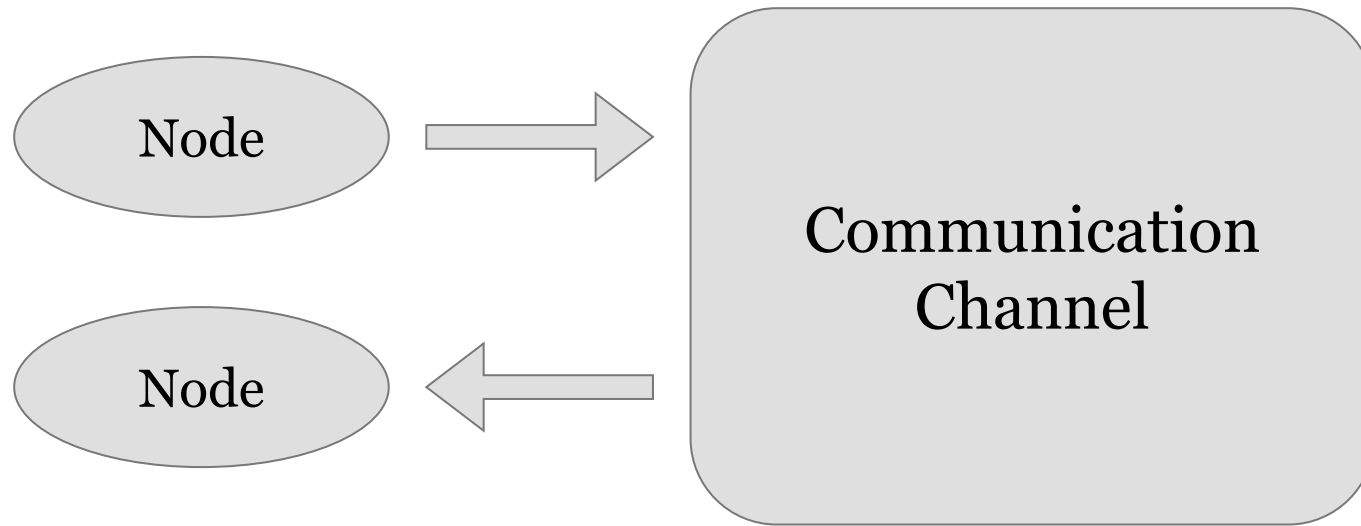


Distributed Publish-Subscribe Middleware

ROS

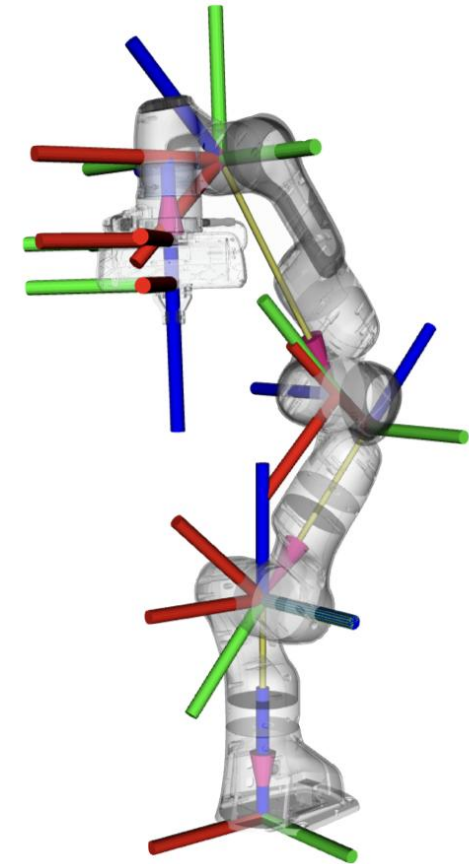


When should you use ROS ?



Use **ROS** when building distributed robotic systems where sensors, actuators, and algorithms must exchange **spatially referenced data** and maintain consistent **coordinate-frame transformations** (e.g., perception, localization, kinematics).

ROS



Robot Operating System (ROS)

What is/isn't ROS ?

Robot Operating System (ROS) was often mistaken as an operating system due to its misleading name.

ROS is a software framework for developing systems that consists of a cluster of computers.

It is often used with a robotics hardware that have

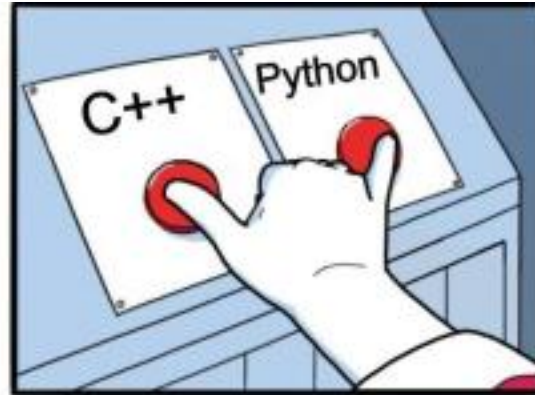


What does ROS do ?

Tools, Standards,
Conventions (REP)

Resource Sharing
Platform

Supported Programming Language for ROS



Common type of information used in robotics



ROS Message Types

[Accel](#)

[Inertia](#)

[Point](#)

[Point32](#)

[Pose](#)

[Pose2D](#)

[Quaternion](#)

[Transform](#)

[Twist](#)

[Vector3](#)

[Wrench](#)

[BatteryState](#)

[CameraInfo](#)

[CompressedImage](#)

[Image](#)

[Imu](#)

[JointState](#)

[Joy](#)

[LaserScan](#)

[PointCloud](#)

[Temperature](#)

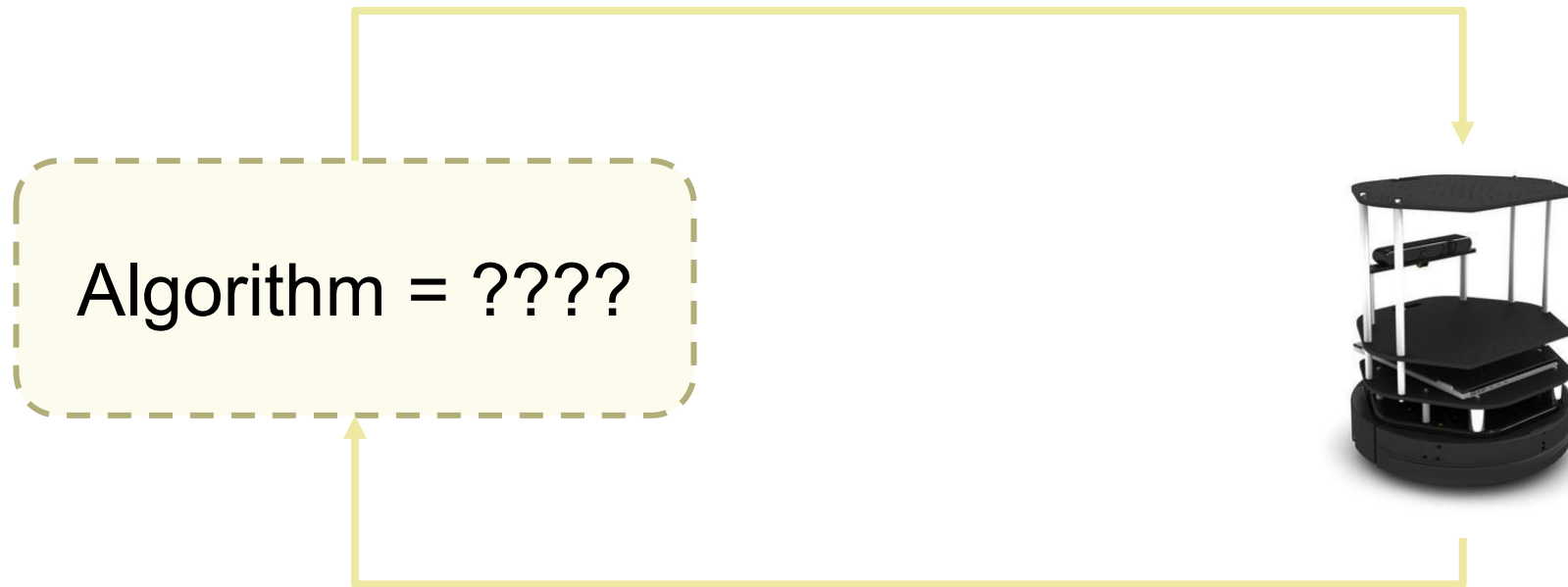
Packages for/by the ROS Community

If your idea is simple, it is most likely to exist as a ROS package.

global_planner rviz gazebo aruco_detect

map_server camera_calibration ros_control

Physical System vs Simulation



Physical System vs Simulation

Simulation-in-the-loop Test



Physical System vs Simulation



Physical System vs Simulation

Tuning with the simulation



Physical System vs Simulation



Physical System vs Simulation



Q1.) Should they use the same computer ?

Physical System vs Simulation



Q2.) Which one should be more powerful ?

System Requirement for ROS

Hardware

- Graphic Card for simulator
- >1.6 GHz, Dual-core (Intel i5 processor or better)
- 4 GB RAM

Operating System

- Ubuntu, Fedora
- ***Windows 10,11 (some bug still exists in old distro)



ROS2

Issues from using ROS(1)

ROS was not designed to support a system of multiple robots due to its use of a single master node.

The paradigm of communication was not designed to support real time operation,

ROS was designed so that the resources should be local (contained within the computer) for optimal performance.

ROS was essentially a centralized system, which is not designed to support multi-agent system.

ROS2



Utilize Data Distribution instead of TCPROS

No Central Master.

Real-time Capability (deterministic execution,
executor-based scheduling)

Distribution (Distro)

The LTS distro is released in even number years (i.e. 2022,2024, 2026,...)

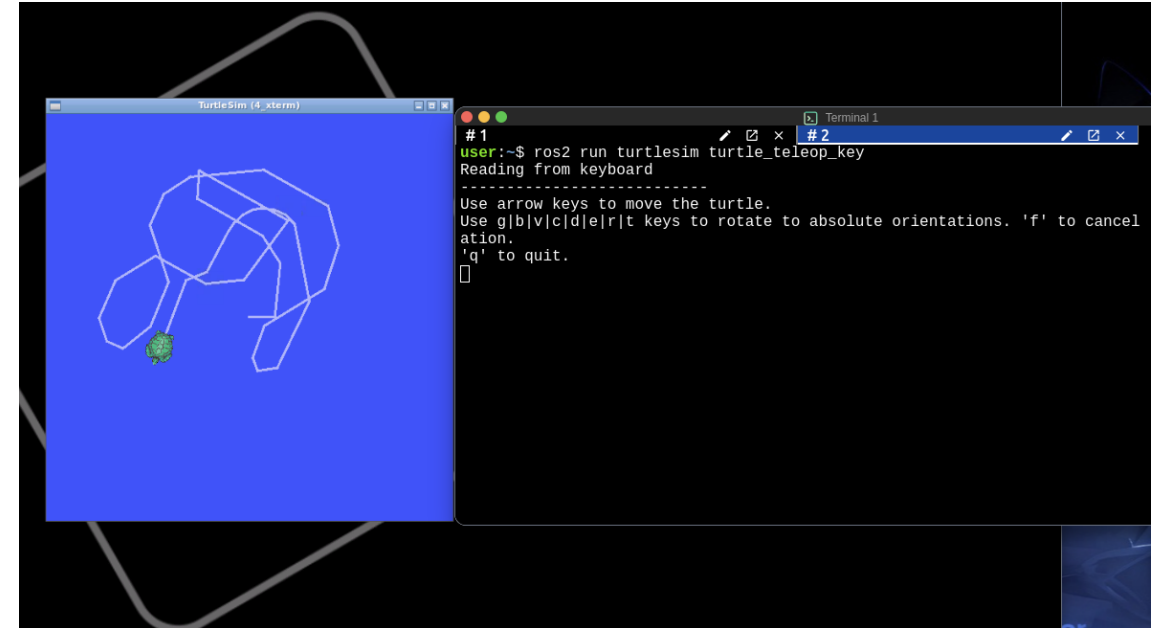
Distro	Release date	Logo	EOL date	ROS Boss
Lyrical Luth	May 22, 2026		May 2031	Shane Loretz
Kilted Kaiju	May 23, 2025		December 2026	Scott K Logan
Jazzy Jalisco	May 23, 2024		May 2029	Marco A. Gutiérrez
Iron Irwini	May 23, 2023		December 4, 2024	Yadunund Vijay
Humble Hawksbill	May 23, 2022		May 2027	Christophe Bédard / Audrow Nash
Galactic Geochelone	May 23, 2021		December 9, 2022	Scott Logan
Foxy Fitzroy	June 5, 2020		June 20, 2023	Jacob Perron / Dharini Dutia

The Construct (Emulating ROS2 environment)

1. Go to <https://app.theconstruct.ai/>
2. Register a new free account or log in with your Google account
3. Click on “My Rosjects” folder then “Create a New Rosject”
4. Name your Rosject, select the desired Distro (Jazzy), and select blank simulation

In the first terminal, run this command

```
ros2 run turtlesim turtlesim_node
```



In a new terminal , run this command

```
ros2 run turtlesim  
turtle_teleop_key
```

Ubuntu : Common Command

```
ls - directory listing
ls -al - formatted listing with hidden files
cd dir - change directory to dir
cd - change to home
pwd - show current directory
mkdir dir - create a directory dir
rm file - delete file
rm -r dir - delete directory dir
rm -f file - force remove file
rm -rf dir - force remove directory dir *
cp file1 file2 - copy file1 to file2
cp -r dir1 dir2 - copy dir1 to dir2; create dir2 if it
doesn't exist
```

List all installed ROS package

```
>> apt list ros-humble* --installed
```

Install a ROS package

```
>> sudo apt install ros-humble-[PACKAGE]
```

Open file explorer from the current working directory

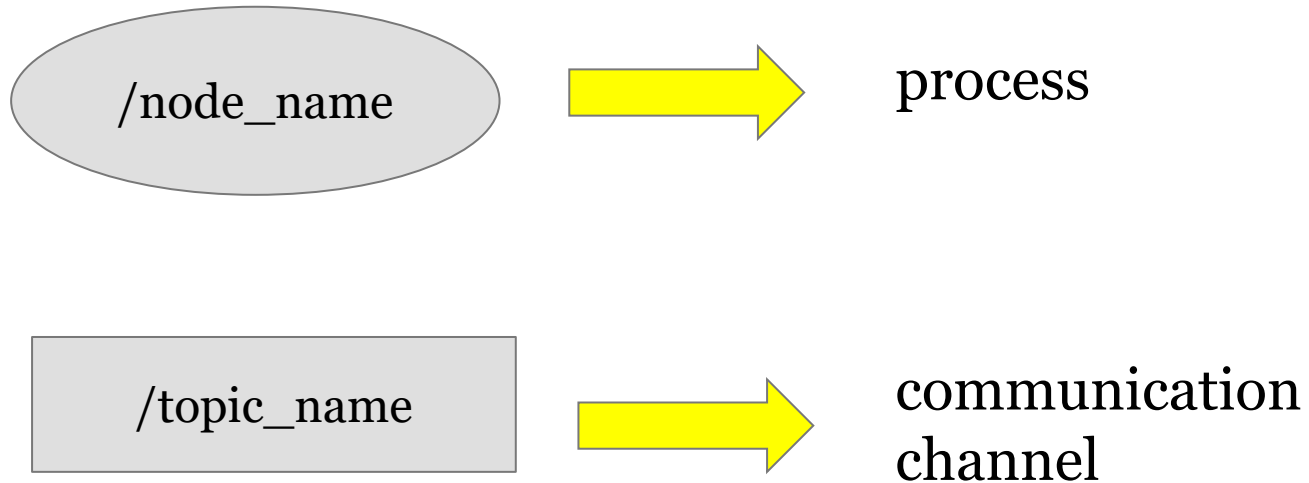
```
>> nautilus .
```

Modify bashrc

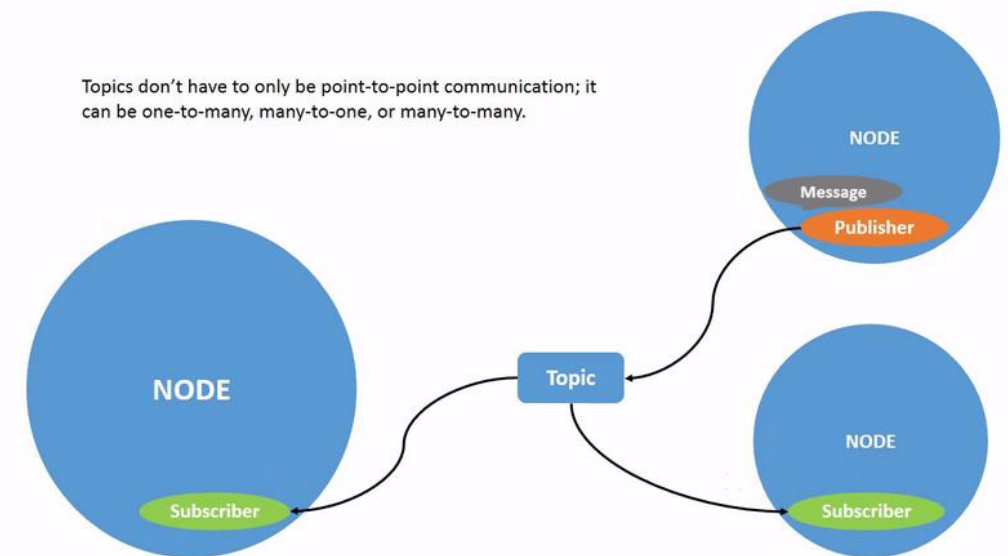
```
>> code ~/.bashrc
```

Nodes & Topics

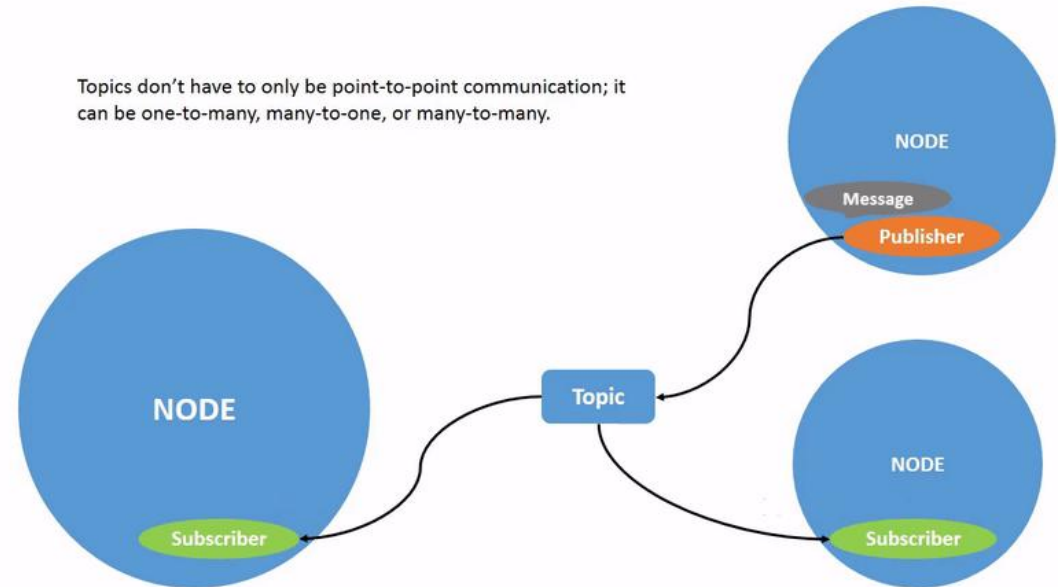
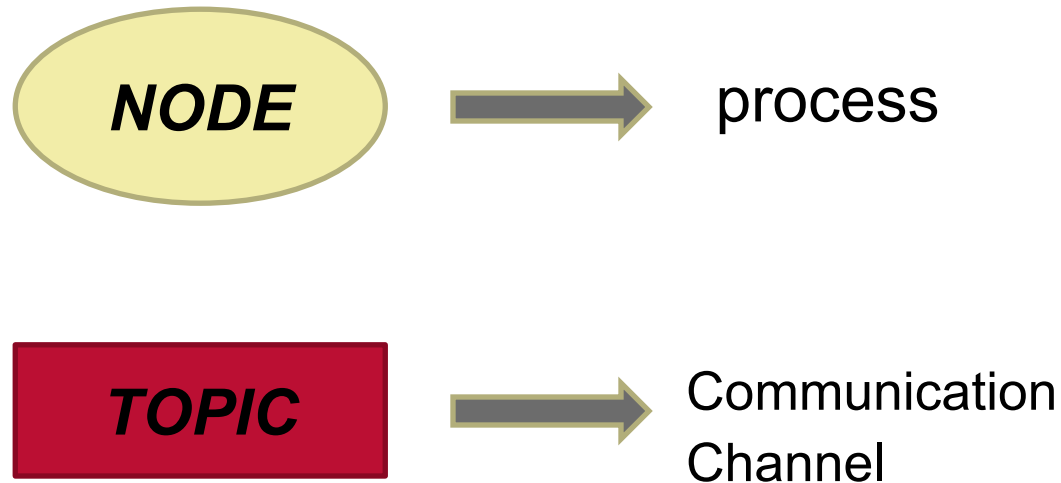
The concept of Nodes and Topics



Node can have a publisher that publish a message to a topic.
Node can also have a subscription that subscribe a message from a topic.



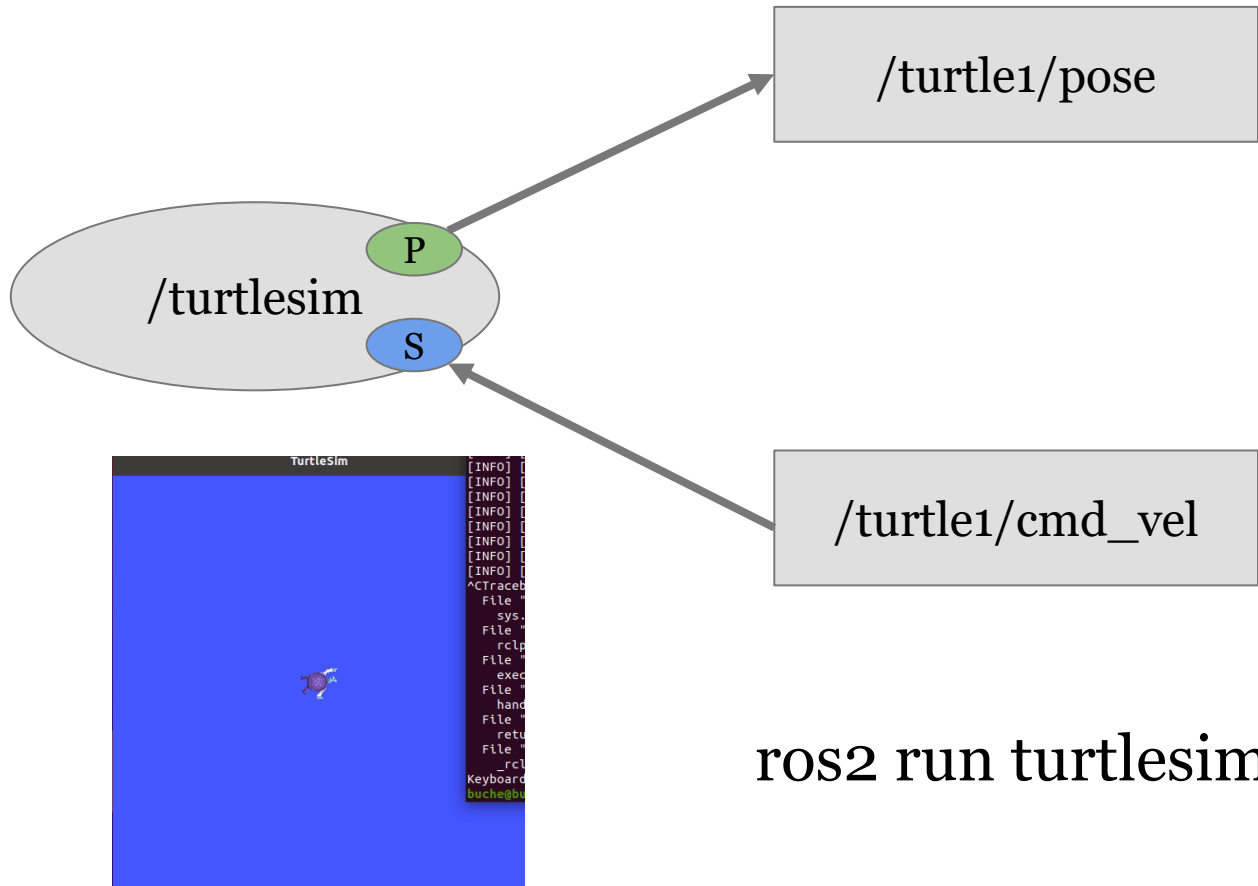
The concept of Nodes and Topics



Q1.) Can a node have both a publisher and a subscription to the same topic ?

Q2.) Should a node have both a publisher and a subscription to the same topic ?

Example: Turtlesim Simulation

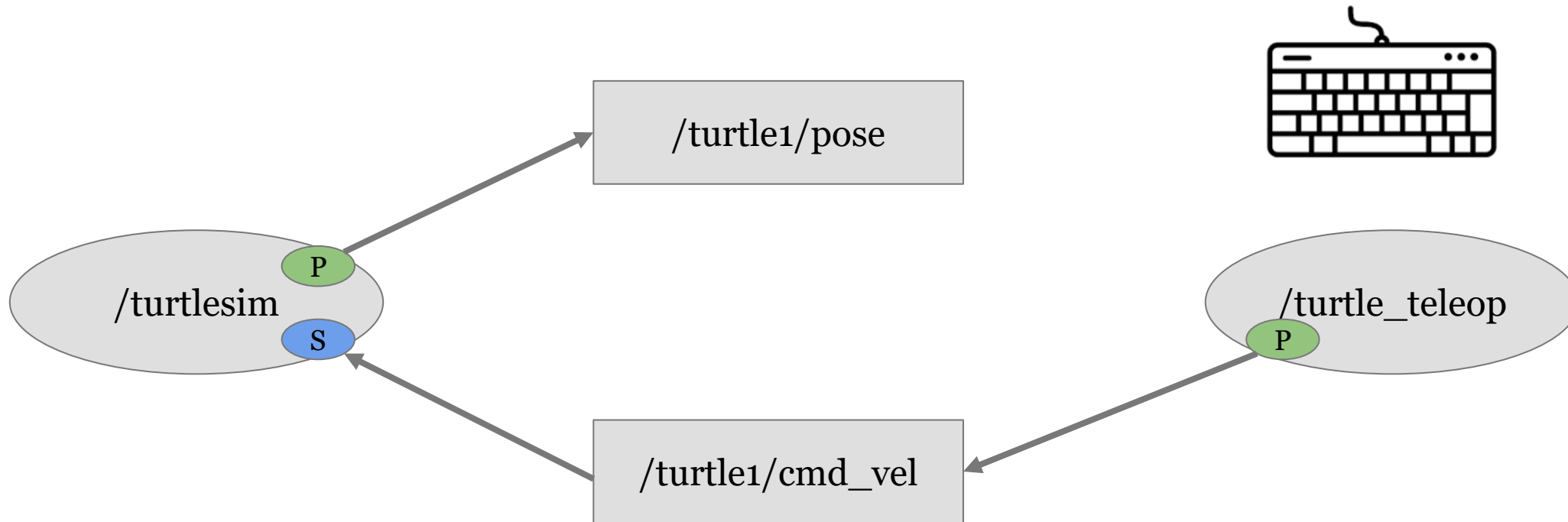


Command lines for nodes:

```
ros2 run [package_name] [executable]
ros2 node list
ros2 node info [node_name] --verbose
ros2 topic list
```

```
ros2 run turtlesim turtlesim_node
```

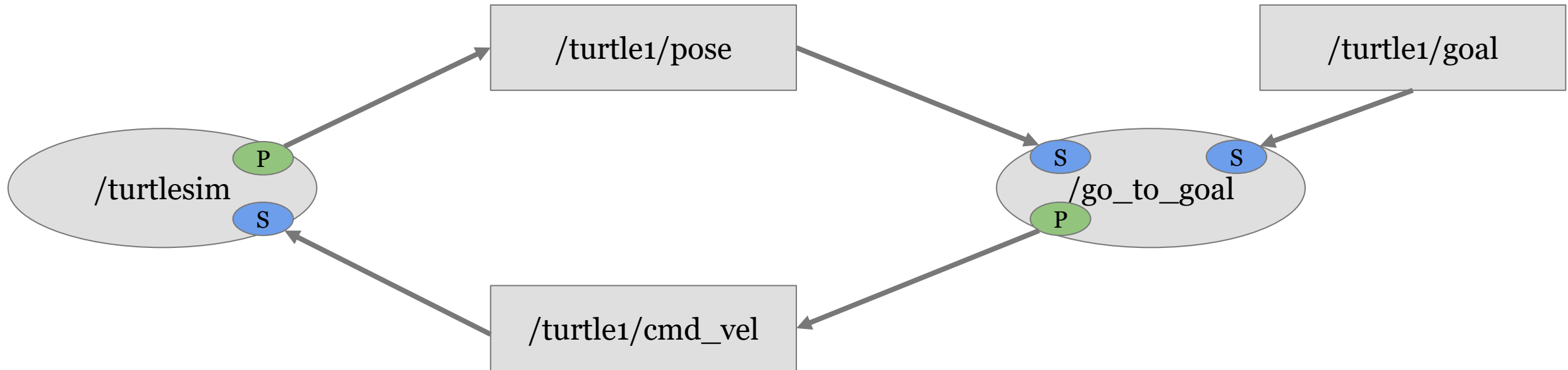
Example: Turtlesim Simulation + Teleoperation



`ros2 run turtlesim turtlesim_node`

`ros2 run turtlesim
turtle_teleop_key`

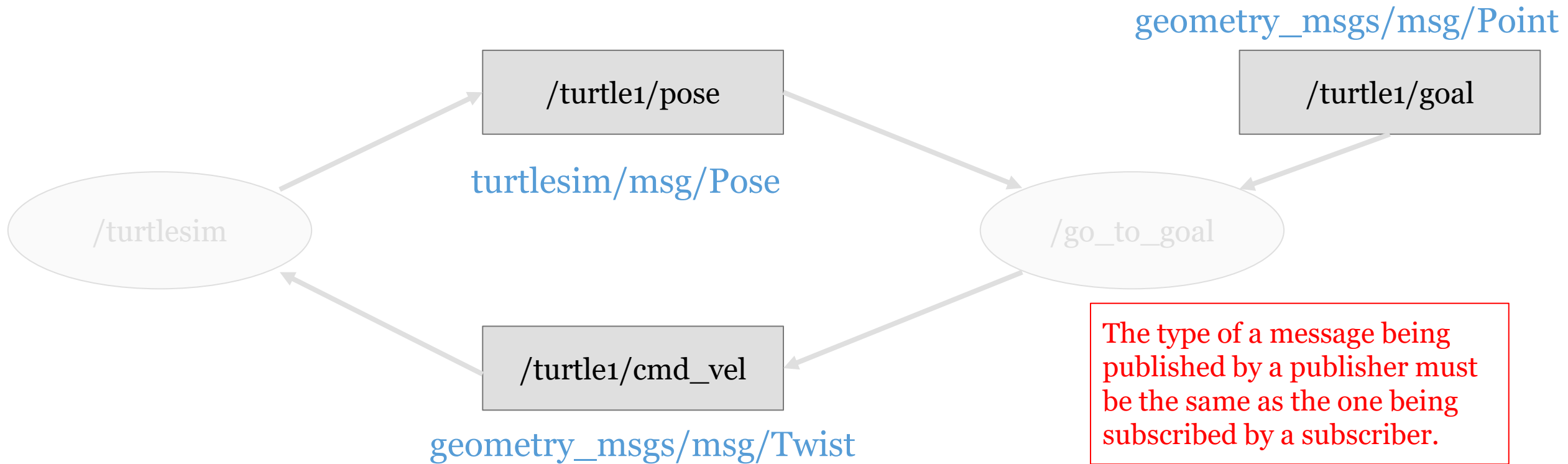
Example: Turtlesim Simulation + Feedback Control



`ros2 run turtlesim turtlesim_node`

`ros2 run turtlesim_control go_to_goal.py`

Message Type



A topic must be consistent throughout the network. Its type can be based on a message type from an existing package or a custom message type.

Message Type

```
cmd_vel: geometry_msgs/msg/Twist:
  linear: geometry_msgs/msg/Vector3
    x: float
    y: float
    z: float
  angular:
    geometry_msgs/msg/Vector3
      x: float
      y: float
      z: float
```

Command lines for topics:

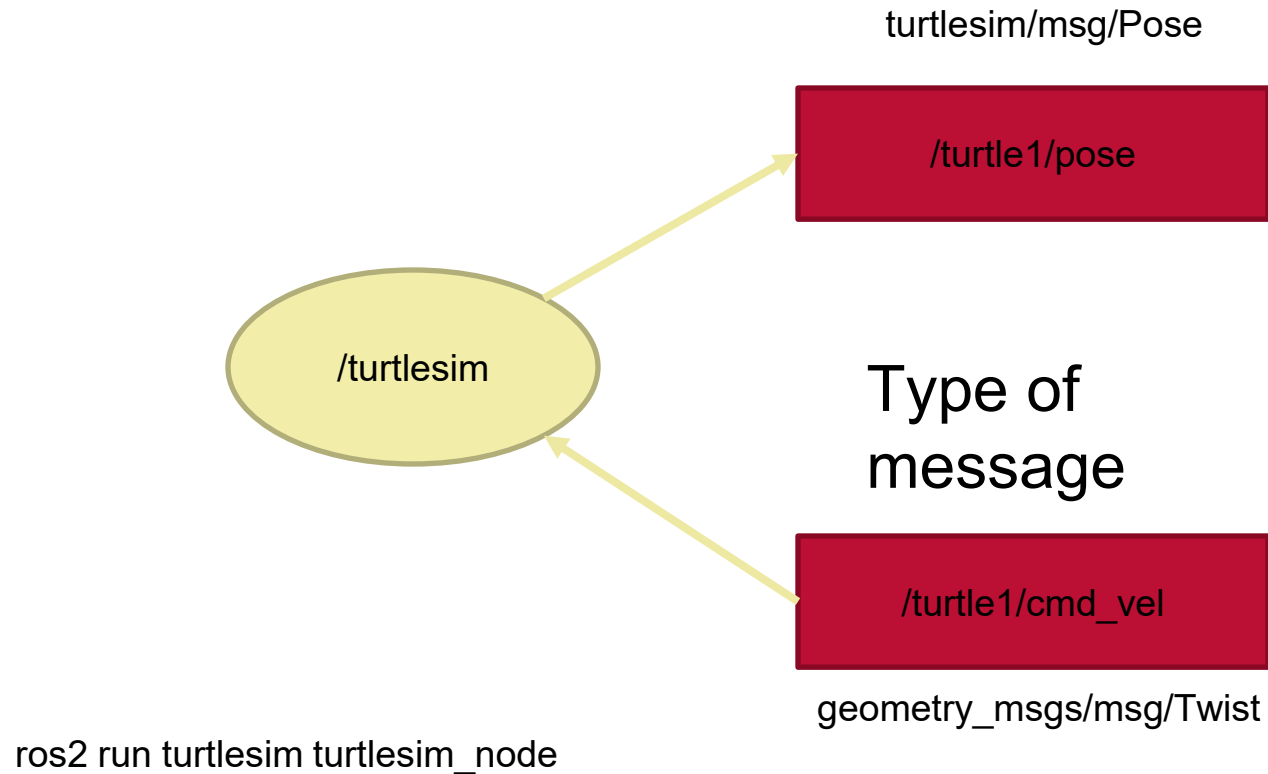
```
ros2 topic echo [topic name]
ros2 topic info [topic name]
ros2 topic pub -r [rate] [message type] [topic_name] [message info]
```

Example Message: Pose

```
pose: geometry_msgs/msg/Pose:  
  orientation: geometry_msgs/msg/Quaternion  
    x : float  
    y: float  
    z: float  
    w: float  
  position: geometry_msgs/msg/Vector3  
    x : float  
    y: float  
    z: float
```

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix} \rightarrow (\mathbf{q}, \mathbf{p}) = \left(\begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}, \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \right)$$

The concept of message



```

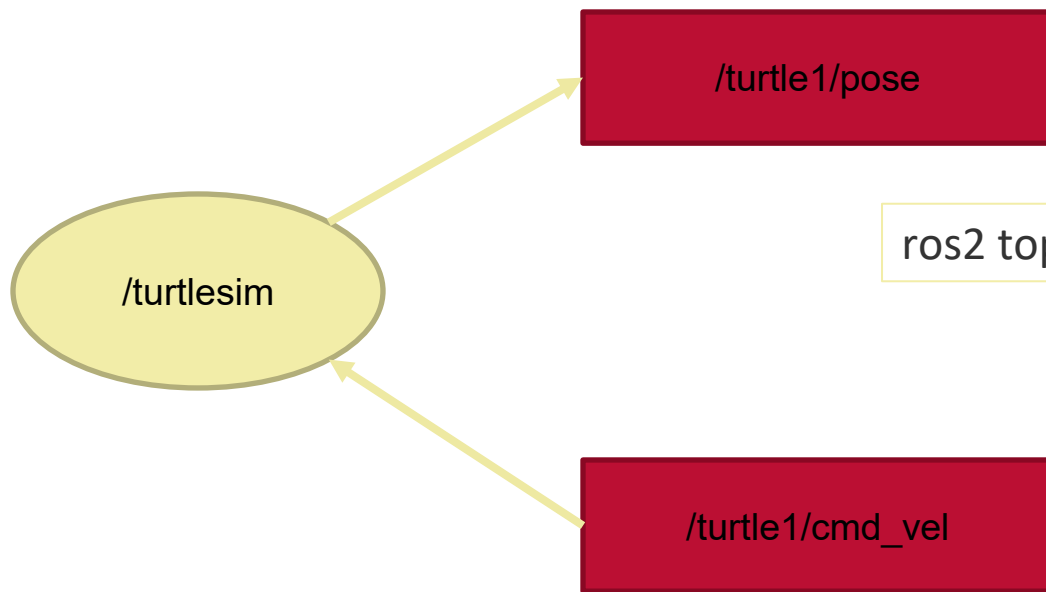
ros2 topic echo [topic_name]
ros2 topic info [topic_name]
ros2 interface show [message_type]

```

Twist :

- linear:
 - x
 - y
 - z
- angular:
 - x
 - y
 - z

Publishing messages from the terminal

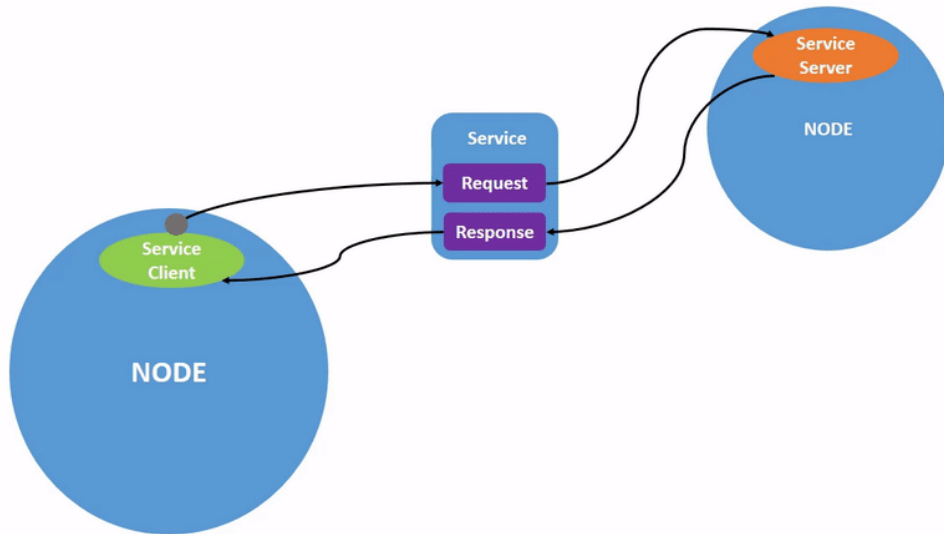


```
ros2 topic pub -r [rate in Hz] [topic_name] [message_type] "[data]"
```

YAML format

```
ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist {linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}
```

The concept of service

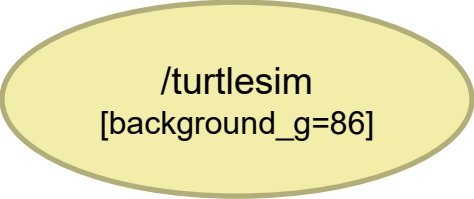


```
ros2 service list
ros2 service info [service_name]
ros2 interface show [service_type]
ros2 service call [service_name] [service_type] [request_data]
```

- 1-1 communication
- Suitable for more private communication that happens once in a while

Node can have a service server that provides service to a service client from other nodes.

The concept of ROS2 Parameter

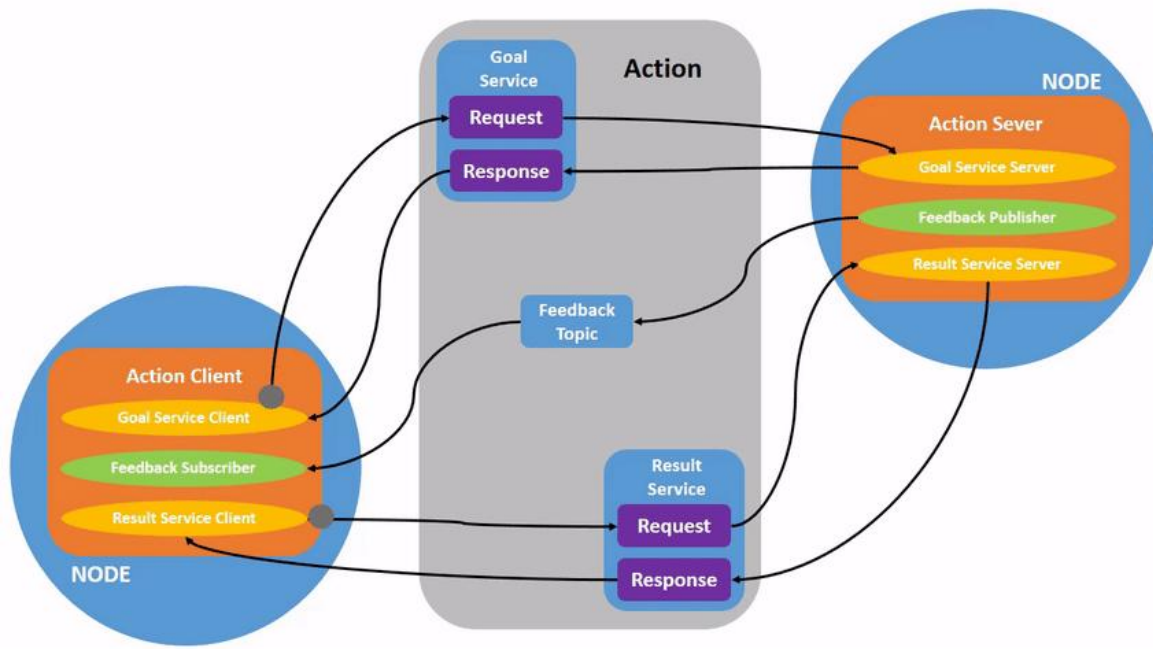


/turtlesim
[background_g=86]

```
ros2 param list
ros2 param get [node_name] [parameter_name]
ros2 param set [node_name] [parameter_name] [value]
ros2 param dump [node_name]
ros2 param load [node_name] [parameter_file]
ros2 run [package_name] [executable] --ros-args --params-file [file_name]
```

Parameter is a constant that is associated with that particular node. It can be modified and accessed by terminal interface.

The concept of action

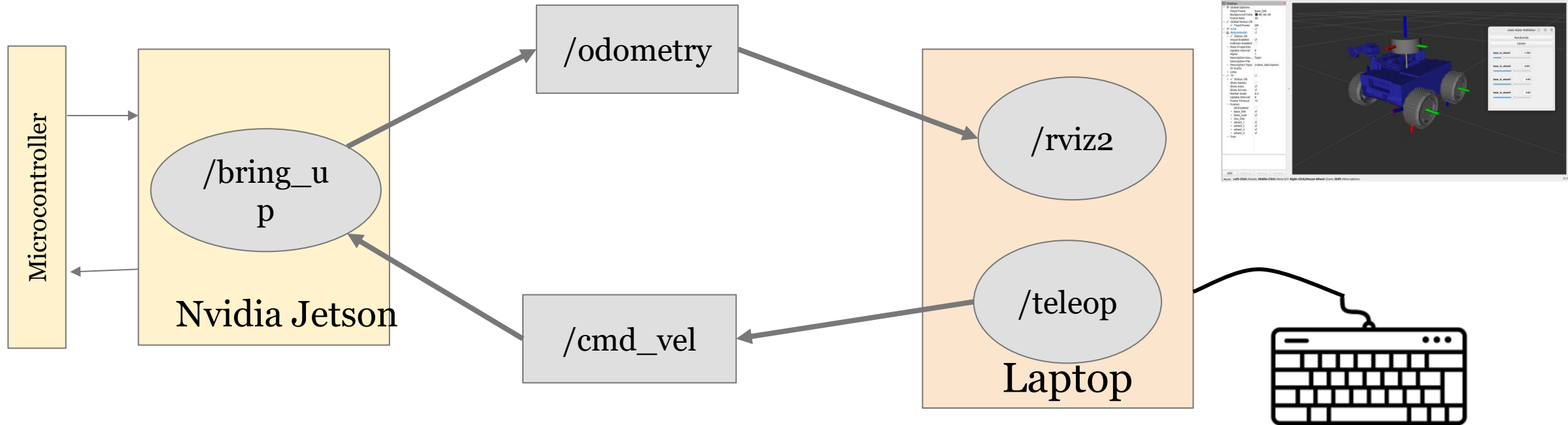


```
ros2 action list
ros2 action info [action_name]
ros2 interface show [action_type]
ros2 action send_goal [action_name] [action_type] [value] -f
```

- 1-1 communication
- Suitable for task that requires time to complete such as planning, optimizing, navigating, etc.

A node can have an action server that execute an "action", then return the result to the client.

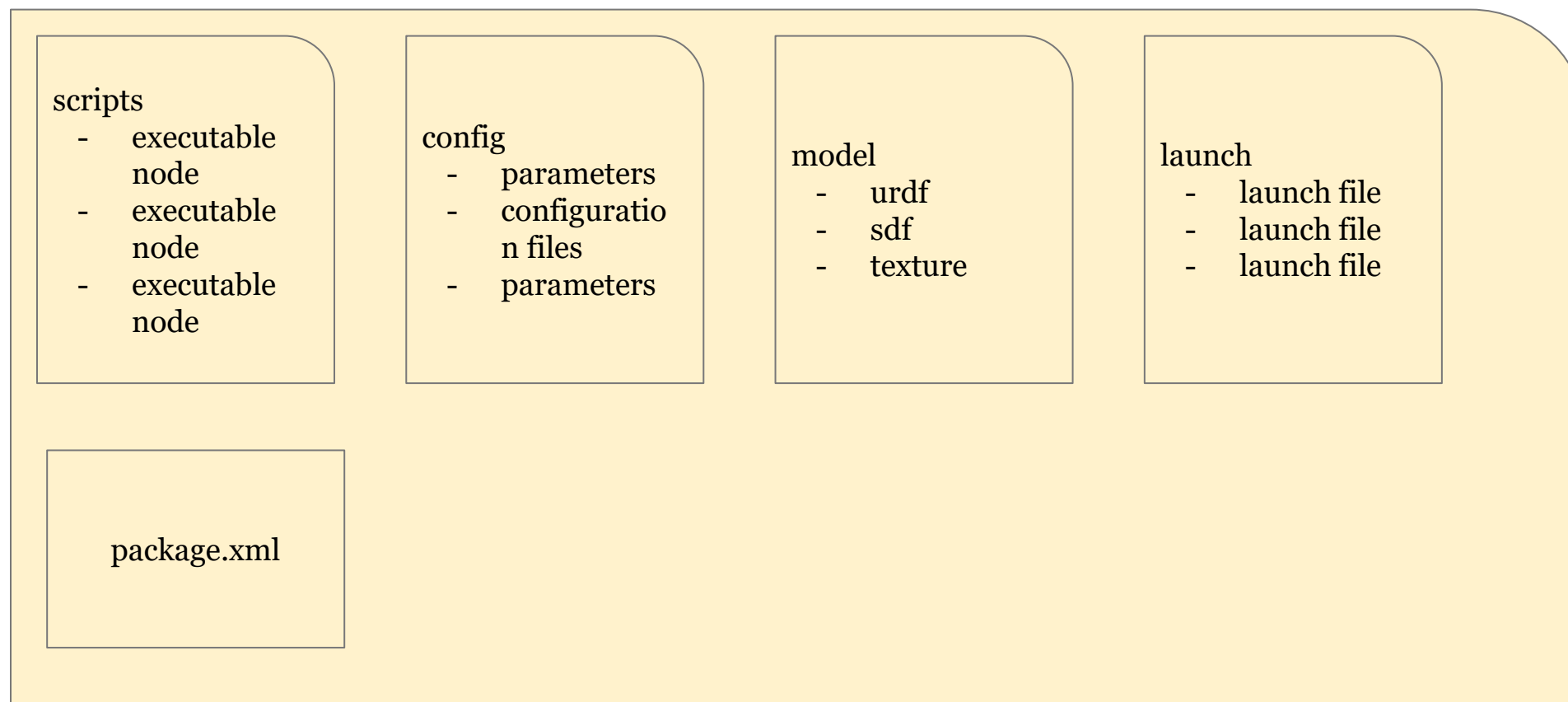
Example: ROS2 Network with Multiple Computers



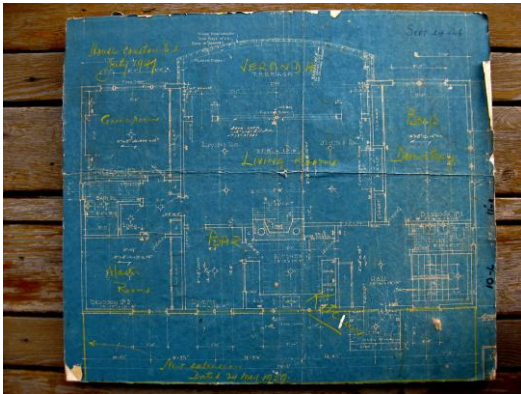
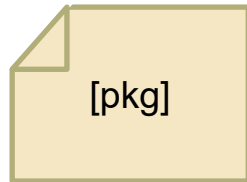
```
ros2 run turtlesim turtlesim_node
```

```
ros2 run turtlesim  
turtle_teleop_key
```

ROS2 Package



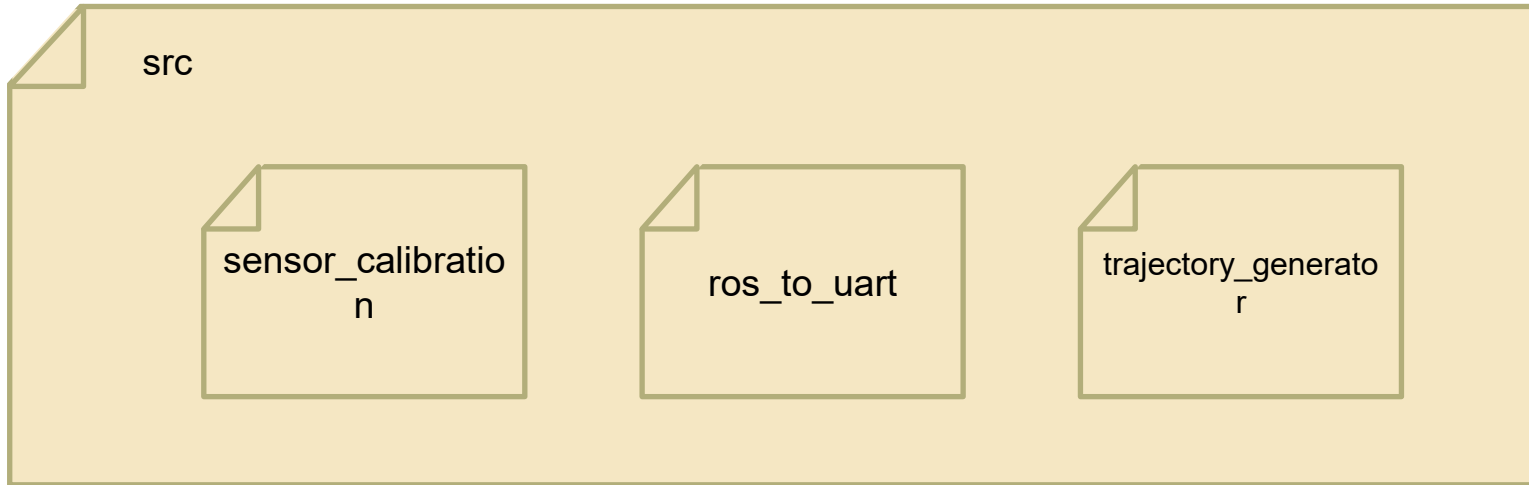
The concept of Package



Package is a collection of organized files in a form of directory, which will be used to "synthesize" nodes, launch system, etc. (usually for a specific task)

- An analogy of a package would be a folder where we can keep all blue prints of a house but not the house itself.

Source folder (src)



"src" directory is where we put all custom packages together.

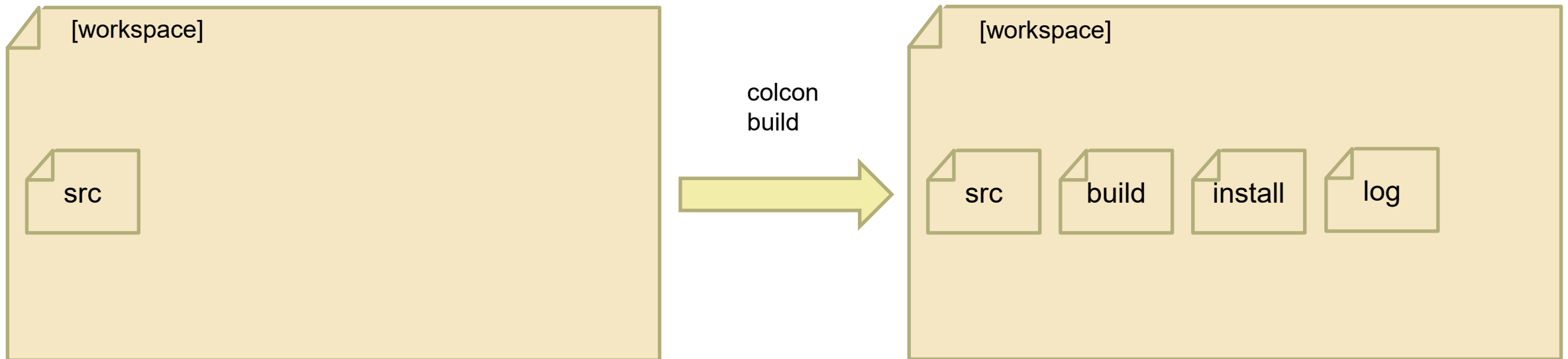
A package can be put inside another folder, which can be referred as a meta-package.

The concept of Workspace



[ROS2] workspace is where we put every custom packages for a project.

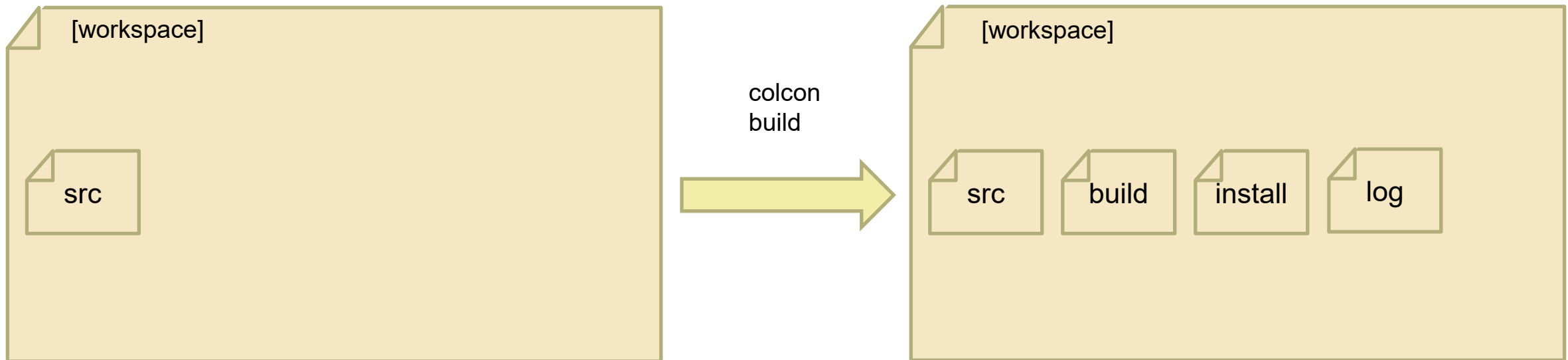
Build System



"colcon build" command will build every packages in the source directory and generate 3 additional directories. When modify "src", always re-build these 3 directories. (with an exception of using symlink install)

Build System

You must not create workspace inside another workspace



"Install" directory is the location of codes that will be used by the ROS2 system. Therefore, only modifying "src" will not change the behavior of your system in run time.

Creating a new workspace

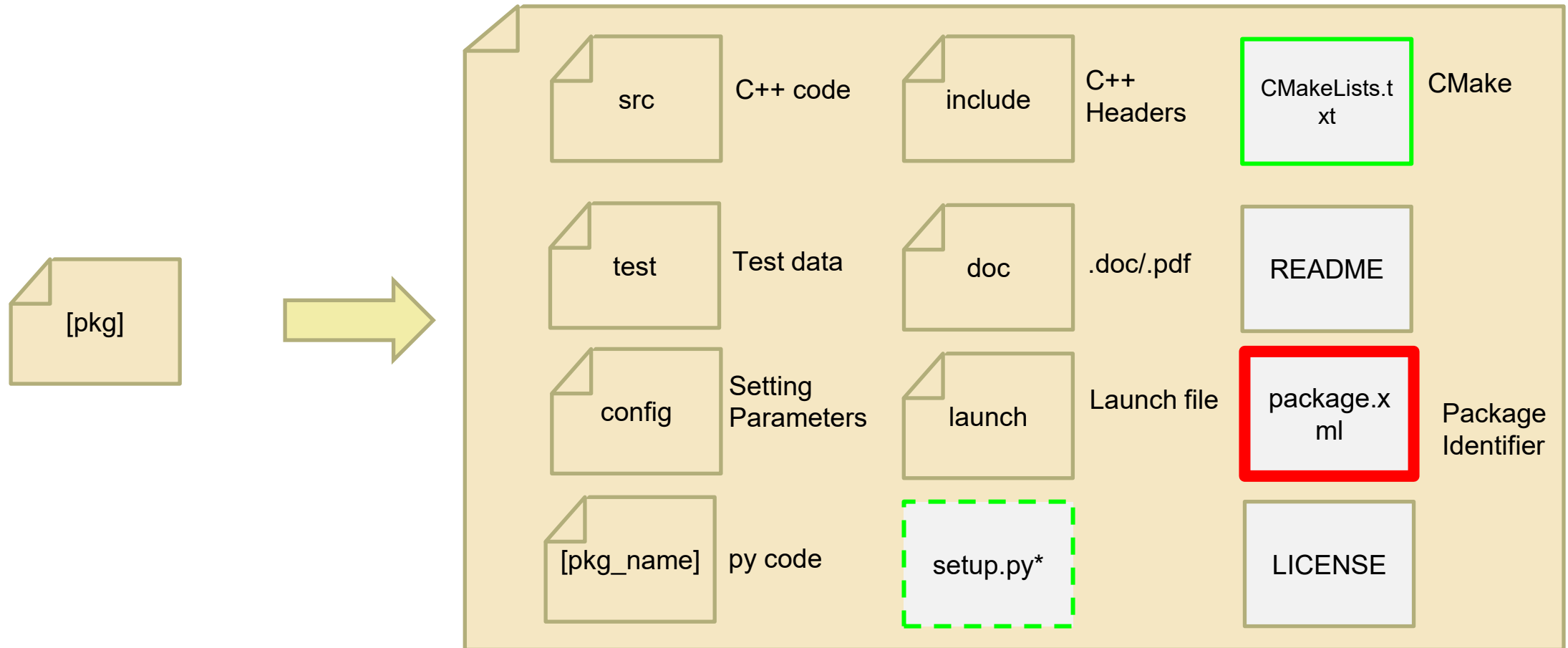
Create and build a new workspace

```
>> mkdir -p ~/[xxx]_ws/src  
>> cd ~/[xxx]_ws  
>> colcon build
```

Adding workspace to .bashrc

```
source ~/[xxx]_ws/install/local_setup.bash
```

Package Layout



<https://automaticaddison.com/organizing-files-and-folders-inside-a-ros-2-package/>

Creating a ROS Package

```
>> cd ~/[xxx]_ws/src
>> ros2 pkg create --build-type ament_python
[package_name]
```

Package with Python

```
>> cd ~/[xxx]_ws/src
>> ros2 pkg create --build-type ament_cmake
[package_name]
```

Package with C++

The package that we create must be in the src directory of our workspace.

What if we have both C++ and Python files ?

Creating my_first_pkg

```
>> cd ~/[xxx]_ws/src
>> ros2 pkg create --build-type ament_python my_first_pkg
>> cd ..
>> colcon build --packages-select my_first_pkg

>> cd src/my_first_pkg
>> code package.xml

>> cd my_first_pkg
>> code my_first_script.py
```

<https://ocs.ros.org/en/diamondback/api/licenses.html>

Node class in ROS Client Library for Python (RCLPY)

rclpy.node.Node
publishers
subscriptions
timers
services
clients
executor
create_publisher
create_subscription
create_timer
create_service
create_client
get_clock
get_logger
get_name

rclpy.publisher.Publisher
topic_name
publish

Subscriber callback
Timer_callback
Service server callback

rclpy.client.Client
-
call
call_async
remove_pending_request
service_is_ready
wait_for_service

General procedure on "running" a node using RCLPY

1. Initialize rclpy using `rclpy.init'`
2. Instantiate a "Node" object
3. Spin node using `rclpy.spin` or spin once in a while-loop
4. If the processs exits the loop, de-construct the node
5. Shut down rclpy using `rclpy.shutdown`

Inheriting a Node class from RCLPY

```
from rclpy.node import Node
```

Import the superclass "Node"

```
class NewNode(Node):
    def __init__(self):
        super().__init__('node_name')
```

"NewNode" inherits from "Node"

```
def main(args=None):
    rclpy.init(args=args)
    node = NewNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

Running a node of class "NewNode"

Attaching a timer to the node

```
class NewNode(Node):
    def __init__(self):
        super().__init__('node_name')
        self.timer = self.create_timer(1.0, timer_callback)
    def timer_callback():
        print('Hello World')
```

Period of the timer

Attaching the callback

Defining callback as a function

Attaching a timer to the node

```
class NewNode(Node):
    def __init__(self):
        super().__init__('node_name')
        self.count = 0
        self.timer = self.create_timer(1.0, self.timer_callback)
    def timer_callback(self):
        print(self.count)
        self.count = self.count + 1
```

Defining new attribute

Make a value persist by accessing the attribute of the instance

Defining callback as a method of the class

Attaching a publisher and publishing a message in a timer

```
from std_msgs.msg import Int32
```

Import type of message

```
class NewNode(Node):
```

```
    def __init__(self):
```

```
        super().__init__('node_name')
```

```
        self.count = 0
```

```
        self.timer = self.create_timer(1.0, self.timer_callback)
```

```
        self.pub = self.create_publisher(Int32, 'output', 10)
```

create a publisher

```
    def timer_callback(self):
```

```
        self.count = self.count + 1
```

```
        msg = Int32()
```

Instantiate a new message

```
        msg.data = self.count
```

```
        self.pub.publish(msg)
```

publish a message

Code for Exercise : drive turtle in a circle

Create this file:

```
cd ~/me640_ws/src/turtlesim_control/turtlesim_control
code circle_controller.py
```

Paste:

Now edit setup.py:

```
cd ~/me640_ws/src/turtlesim_control
code setup.py
```

Find entry_points and add:

```
entry_points={
'console_scripts': [
'circle_controller = turtlesim_control.circle_controller:main',
],
},
```

Build and run:

```
cd ~/me640_ws
colcon build --packages-select turtlesim_control --symlink-install
source install/setup.bash
```

Terminal 1:

```
ros2 run turtlesim turtlesim_node
```

Terminal 2:

```
ros2 run turtlesim_control circle_controller
```

```
#!/usr/bin/env python3
```

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
```

```
class CircleController(Node):
    def __init__(self):
        super().__init__('circle_controller')

        self.pub = self.create_publisher( Twist, 'turtle1/cmd_vel',10 )

        self.timer = self.create_timer(0.1, self.timer_callback)

        self.get_logger().info('Circle controller node has started.')

    def timer_callback(self):
        msg = Twist()

        msg.linear.x = 2.0
        msg.angular.z = 1.0

        self.pub.publish(msg)
```

```
def main(args=None):
    rclpy.init(args=args)

    node = CircleController()
    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

Code for Exercise : turtle2 follow turtle1

- **Terminal 1: Start turtlesim**

```
ros2 run turtlesim turtlesim_node
```

- **Terminal 2: Spawn turtle2**

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: 'turtle2'}"
```

- **Terminal 3: Control turtle1 using keyboard**

```
ros2 run turtlesim turtle_teleop_key
```

Keep this terminal active when using the arrow keys.

- **Terminal 4: Run follower node**

```
ros2 run turtlesim_controller turtle_follower
```

- **Inside entry_points, add:**

```
'turtle_follower = turtlesim_controller.turtle_follower:main',
```

- **So, it may look like this:**

```
entry_points={  
    'console_scripts': [  
        'turtle_follower = turtlesim_controller.turtle_follower:main',  
    ],  
},
```

Build and source

```
cd ~/workspace_name
```

```
colcon build --packages-select turtlesim_controller --symlink-install
```

```
source install/setup.bash
```

Links

- Github: https://github.com/tchoopojcharoen/simple_exercise/tree/rcipy_tutorial
- Slides:
 - 1) https://github.com/tchoopojcharoen/ROS2_pi_uwaterloo/blob/main/ROS2_pi_01.pdf
 - 2) https://docs.google.com/presentation/d/1ljf_5OKg6Fc9qIX2WaIG2sxCiyDxQGLgppOHNE3uEas/edit?slide=id.p...